

EchoMind 重点代码讲解

前言

有需要的可以下载pdf。但是请不要倒卖，这个项目我沉淀了很久，定价也不是很高，感谢🙏（最后有用的话记得多推荐一下😊）

本文档讲解 EchoMind 中最能体现工程设计的关键代码。内容按实际请求链路组织，而不是按文件顺序罗列。

重要说明：EchoMind 当前代码中已经实现端到端评测链路，核心文件为 evaluation/evaluator.py，HTTP 入口为 api/main.py 中的 POST /eval/run。评测不是简单 mock，而是真实调用 AgentOrchestrator.run() 生成回复，再用 LLM-as-Judge 从相关性、准确性、完整性、有用性四个维度评分，并支持意图识别 Accuracy/Macro-F1、回归检测和优化建议。

1. /chat 主链路

文件：api/main.py

/chat 是系统主入口，完整串起记忆、知识库检索、意图识别、Agent 路由、回复生成和画像更新。

代码块

```
1  @app.post("/chat", response_model=ChatResponse)
2  async def chat(req: ChatRequest):
3      if _orchestrator is None or _memory is None:
4          raise HTTPException(503, "服务未就绪")
5
6      conv_id = req.conv_id or str(uuid.uuid4())
7
8      # 1. 读取记忆上下文
9      mem_ctx = await _memory.get_context(req.user_id, conv_id,
10                                         query=req.message)
11
12     # 2. 取最近对话，给意图识别使用
13     history = [
14         {"role": m.role.value, "content": m.content}
15         for m in mem_ctx.recent_messages[-5:]
16     ] if mem_ctx.recent_messages else None
17
18     # 3. 检索知识库，并和记忆上下文一起拼入 Agent 背景
```

```

18     knowledge_text, knowledge_used = await
_build_knowledge_context(req.message)
19     context_parts = [mem_ctx.to_prompt_text()]
20     if knowledge_text:
21         context_parts.append(knowledge_text)
22     full_context = "\n\n".join(part for part in context_parts if part)
23
24     # 4. 构造 Orchestrator 请求
25     orch_req = OrcReq(
26         message=req.message,
27         user_id=req.user_id,
28         conv_id=conv_id,
29         context=full_context,
30         history=history,
31     )
32
33     # 5. 执行 Agent 编排
34     result = await _orchestrator.run(orch_req)
35
36     # 6. 写入工作记忆
37     await _memory.add_message(req.user_id, conv_id, MsgRole.USER, req.message)
38     await _memory.add_message(req.user_id, conv_id, MsgRole.ASSISTANT,
result.response)
39
40     # 7. 异步更新用户画像，不阻塞响应
41     asyncio.create_task(_memory.update_profile(req.user_id, conv_id))

```

关键点

步骤	作用
get_context()	一次性读取工作记忆、情景记忆、用户画像和摘要
history	只取最近 5 条，控制意图识别 prompt 长度
_build_knowledge_context())	调用知识库检索、查询改写、并行召回和重排
context=full_context	把三级记忆和 Top-K 知识库结果拼成 Agent 可读背景信息

add_message()	用户消息和助手回复都写入 Redis 工作记忆
update_profile()	后台异步提炼用户偏好，不影响接口延迟

2. 意图识别：三路融合 + 加权投票

文件: core/intent_recognizer.py

主识别流程

代码块

```
1  async def recognize(self, message: str, history=None) -> IntentResult:
2      key = self._cache_key(message)
3      if key in self._cache:
4          self.cache_hits += 1
5          return self._cache[key]
6      self.cache_misses += 1
7
8      t0 = time.monotonic()
9
10     llm_task = asyncio.create_task(self._llm_recognize(message, history))
11     emb_task = asyncio.create_task(self._embedding_recognize(message)) if
self._embedding_enabled else None
12     pat = self._pattern_recognize(message)
13
14     if emb_task:
15         llm, emb = await asyncio.gather(llm_task, emb_task)
16     else:
17         llm = await llm_task
18         emb = {"intent": IntentCategory.OTHER, "confidence": 0.0}
19
20     intent = self._vote(llm, emb, pat)
21     entities = await self._extract_entities(message)
22     urgency = self._urgency(message, intent)
```

为什么这样写

代码	设计意义

asyncio.create_task()	LLM 和 Embedding 同时开始，避免串行等待
pat = self._pattern_recognize()	关键词匹配是本地 CPU 操作，零网络延迟
asyncio.gather()	总耗时接近最慢的那一路，而不是两路相加
_vote()	三路结果统一成一个最终意图
_extract_entities()	意图之外额外提取订单号、金额、错误码等实体

加权投票

代码块

```
1 def _vote(self, llm: Dict, emb: Dict, pat: Dict) -> IntentCategory:
2     if self._embedding_enabled:
3         weights = [(llm, 0.7), (emb, 0.2), (pat, 0.1)]
4     else:
5         weights = [(llm, 0.85), (pat, 0.15)]
6
7     scores: Dict[IntentCategory, float] = {}
8     for result, w in weights:
9         cat = result.get("intent", IntentCategory.OTHER)
10        conf = result.get("confidence", 0.0)
11        scores[cat] = scores.get(cat, 0.0) + w * conf
12
13    best = max(scores, key=scores.get)
14    return best if scores[best] >= self.threshold else IntentCategory.OTHER
```

权重策略：

模式	权重
官方 API 模式	LLM 70% + Embedding 20% + Pattern 10%
第三方兼容 API 模式	LLM 85% + Pattern 15%，禁用 Embedding

Embedding 兜底

当前 Anthropic SDK 没有 embeddings 资源，所以代码做了远端优先、本地兜底：

代码块

```
1 async def _embed_text(self, text: str) -> List[float]:
```

```

2     embeddings = getattr(self.client, "embeddings", None)
3     if embeddings is not None:
4         try:
5             resp = await embeddings.create(model="voyage-3-lite", input=[text])
6             return list(resp.data[0].embedding)
7         except Exception as ex:
8             logger.warning(f"远端 Embedding 失败，使用本地向量兜底：{ex}")
9
10    return self._local_embedding(text)

```

本地向量是稳定的字符 n-gram 哈希向量：

代码块

```

1    @staticmethod
2    def _local_embedding(text: str, dims: int = 256) -> List[float]:
3        normalized = text.lower().strip()
4        vec = [0.0] * dims
5        tokens = set()
6        for n in (1, 2, 3):
7            if len(normalized) >= n:
8                tokens.update(normalized[i:i + n] for i in range(len(normalized) -
9                                n + 1))
10
11        for token in tokens:
12            digest = hashlib.md5(token.encode("utf-8")).digest()
13            idx = int.from_bytes(digest[:4], "big") % dims
14            sign = 1.0 if digest[4] % 2 == 0 else -1.0
15            vec[idx] += sign
16        return vec

```

这个设计保证了“Embedding 路”不会因为 SDK 缺能力而完全不可用。

3. MCP 工具调用：缓存、熔断、fallback

文件：mcp/[tool_manager.py](#)

工具调用生命周期

代码块

```

1    async def call(self, name, params, context=None, *, use_cache=True,
2                  rerank_top_k=0):
3        tool = self._tools.get(name)

```

```

3         if not tool:
4             return ToolResult(success=False, data=None, tool_name=name, error=f"工具
            不存在: {name}")
5
6         if use_cache and tool.cache_ttl > 0:
7             cached = self._get_cache(name, params)
8             if cached is not None:
9                 tool.stats.total += 1
10                tool.stats.success += 1
11                return ToolResult(success=True, data=cached, tool_name=name,
            cached=True)
12
13        if not tool.breaker.allow():
14            error = f"工具熔断中: {name}, 请稍后重试"
15            return await self._fallback_result(tool, params, context, error)
16
17        tool.stats.total += 1
18        try:
19            self._validate_params(tool, params)
20            data = await asyncio.wait_for(tool.handler(params, context),
            timeout=tool.timeout_s)
21            ...
22        except asyncio.TimeoutError:
23            ...
24            return await self._fallback_result(tool, params, context, "执行超时")
25        except Exception as ex:
26            ...
27            return await self._fallback_result(tool, params, context, str(ex))

```

重点设计

环节	目的
缓存检查	相同参数直接返回，降低检索和 LLM 重排成本
熔断检查	工具连续失败后短期拒绝请求，避免雪崩
参数校验	使用 JSON Schema 检查 required 和类型
asyncio.wait_for	工具级超时，避免请求无限挂起

统计更新	记录成功率、延迟、连续失败，供 Monitor 使用
fallback	工具失败时返回可解释降级结果

fallback 实现

代码块

```
1  async def _fallback_result(self, tool, params, context, error) -> ToolResult:
2      if tool.fallback is None:
3          return ToolResult(success=False, data=None, tool_name=tool.name,
4                              error=error)
5      try:
6          data = tool.fallback(params, context, error)
7          if asyncio.iscoroutine(data):
8              data = await data
9          return ToolResult(
10              success=True,
11              data=data,
12              tool_name=tool.name,
13              error=error,
14          )
15     except Exception as ex:
16         return ToolResult(
17             success=False,
18             data=None,
19             tool_name=tool.name,
20             error=f"{error}; fallback失败: {ex}",
21         )
```

在 `api/main.py` 中，知识库工具注册了 fallback:

代码块

```
1  def knowledge_fallback(params, context, error):
2      query = params.get("query", "")
3      return [{
4          "title": "知识库降级结果",
5          "content": f"知识库暂时不可用，未能完成对“{query}”的语义检索。请稍后重试，或转人工客服确认。",
6          "score": 0.0,
7          "fallback": True,
8          "error": error,
```

4. 查询改写 + 并行召回 + LLM 重排

文件: `mcp/tool_manager.py`

代码块

```
1  async def search_with_rewrite(self, tool_name, query, top_k=5, context=None) -
    > ToolResult:
2      sub_queries = await self.rewrite_query(query, n=3)
3
4      recall_k = max(top_k, 5)
5      tasks = [
6          self.call(tool_name, {"query": q, "top_k": recall_k}, context,
use_cache=True)
7          for q in sub_queries
8      ]
9      results = await asyncio.gather(*tasks, return_exceptions=True)
10
11     seen, merged = set(), []
12     for r in results:
13         if isinstance(r, ToolResult) and r.success and isinstance(r.data,
list):
14             for item in r.data:
15                 key = hashlib.md5(str(item).encode()).hexdigest()
16                 if key not in seen:
17                     seen.add(key)
18                     merged.append(item)
19
20     if not merged:
21         return ToolResult(success=False, data=[], tool_name=tool_name,
error="所有子查询均无结果")
22
23     reranked = await self._rerank(query, merged, top_k)
24     return ToolResult(success=True, data=reranked, tool_name=tool_name,
reranked=True)
```

这里有一个容易忽略的细节：每个子查询召回 `recall_k = max(top_k, 5)` 条，而不是只召回 `top_k` 条。这样重排阶段有更大的候选池，避免过早截断。

解决的问题

问题	代码策略
单 query 召回不全	rewrite_query() 生成多个角度
多 query 增加延迟	asyncio.gather() 并行召回
重复片段过多	内容 MD5 去重
向量排序不等于答案质量	_rerank() 使用 LLM 重排

5. ChromaDB 知识库

文件: mcp/[knowledge_base.py](#)

初始化

代码块

```
1  try:
2      self._client = chromadb.HttpClient(host=chroma_host, port=chroma_port)
3      self._client.heartbeat()
4      self._use_server = True
5  except Exception:
6      self._client = chromadb.PersistentClient(
7          path=chroma_path,
8          settings=chromadb.Settings(anonymized_telemetry=False),
9      )
10
11  self._collection = self._client.get_or_create_collection(
12      name=self.COLLECTION_NAME,
13      metadata={"description": "EchoMind RAG 知识库"},
14  )
15
16  if self._collection.count() == 0:
17      self._load_default_docs()
```

关键点

设计	作用
----	----

优先 HttpClient	Docker Compose 下连接独立 ChromaDB 服务
失败降级 PersistentClient	本地开发时没有 ChromaDB 服务也能运行
collection 名称 knowledge_base	和记忆模块的 episodic、user_profile 隔离
空库自动导入默认文档	项目启动后可以直接演示检索

检索

代码块

```
1 results = self._collection.query(  
2     query_texts=[query],  
3     n_results=top_k,  
4 )
```

ChromaDB 会对 query_texts 自动生成向量并做语义匹配。返回结果会被转换成：

代码块

```
1 {  
2     "title": meta.get("title", ""),  
3     "content": doc,  
4     "score": round(1.0 - dist, 4),  
5     "chunk": meta.get("chunk_index", 0),  
6 }
```

6. 三级记忆：Redis + ChromaDB

文件：memory/[conversation_memory.py](#)

写入工作记忆

代码块

```
1 async def add_message(self, user_id, conv_id, role, content, metadata=None):  
2     key = self._wm_key(user_id, conv_id)  
3  
4     self._redis.lpush(key, json.dumps({
```

```

5         "role": msg.role.value,
6         "content": msg.content,
7         "ts": msg.timestamp.isoformat(),
8         "metadata": msg.metadata,
9     }))
10    self._redis.expire(key, 86400)
11
12    if self._redis.llen(key) >= self.COMPRESS_AT:
13        await self._compress(user_id, conv_id)

```

Redis key:

代码块

```
1    wm:{user_id}:{conv_id}
```

自动压缩

代码块

```

1    async def _compress(self, user_id, conv_id):
2        messages = await self._get_working_memory(user_id, conv_id)
3        if len(messages) < self.COMPRESS_AT:
4            return
5
6        to_compress = messages[:-5]
7        keep = messages[-5:]
8
9        text = "\n".join(f"{m.role.value}: {m.content}" for m in to_compress)
10       prompt = f"用 2-3 句话总结以下对话的关键信息: \n{text}"
11       ...
12       self._redis.setex(skey, 86400, new_summary)
13       await self._store_episodic(user_id, conv_id, text, summary)

```

压缩后的摘要会写入 ChromaDB collection:

代码块

```
1    episodic
```

用户画像

```
1  async def update_profile(self, user_id: str, conv_id: str) -> None:
2      messages = await self._get_working_memory(user_id, conv_id)
3      text = "\n".join(f"{m.role.value}: {m.content}" for m in messages[-10:])
4
5      prompt = f"""从以下对话中提炼用户偏好和关键实体，返回 JSON。
6  对话：
7  {text}
8
9  返回格式：{{"preferences": ["..."], "entities": {{{"产品": [], "问题类型":
    [...]}}}}"""
```

画像写入 ChromaDB collection：

代码块

```
1  user_profile
```

下一次请求时，get_context() 会读取画像并拼入 prompt：

代码块

```
1  if self.user_profile:
2      parts.append(f"[用户画像]\n{json.dumps(self.user_profile,
    ensure_ascii=True)}")
```

三个存储层

层级	存储	内容
工作记忆	Redis	当前会话最近消息，24h TTL
情景记忆	ChromaDB episodic	压缩后的历史对话摘要
用户画像	ChromaDB user_profile	用户偏好和关键实体

7. Agent 路由：意图映射、性能路由、降级

文件：[agents/agent_orchestrator.py](#)

主流程

```

1 代码块 async def run(self, req: Request) -> OrchestratorResult:
2      if req.intent is None:
3          intent_result = await self._intent_recognizer.recognize(req.message,
4              history=req.history)
5          req.intent = intent_result.intent
6          req.urgency = intent_result.urgency
7
8      collaboration = self._collaboration_targets(req)
9      if len(collaboration) > 1:
10         return await self.run_parallel(req, collaboration)
11
12     agent_type = self._route(req.intent, req.urgency)
13     response = await self._execute(req, agent_type)

```

当前实现里，复合问题检测已经接入主链路。不是只有手动调用 `run_parallel()` 才会并行协作。

意图路由

代码块

```

1  _INTENT_ROUTING = {
2      IntentCategory.TECHNICAL: AgentType.TECHNICAL,
3      IntentCategory.BILLING: AgentType.BILLING,
4      IntentCategory.ACCOUNT: AgentType.BILLING,
5      IntentCategory.ESCALATION: AgentType.ESCALATION,
6  }

```

复合问题检测

代码块

```

1  def _collaboration_targets(self, req: Request) -> List[AgentType]:
2      msg = req.message.lower()
3      targets: List[AgentType] = []
4
5      technical_kws = ["崩溃", "报错", "error", "crash", "无法登录", "登录失败",
6          "500", "401"]
7      billing_kws = ["退款", "扣款", "发票", "账单", "支付", "订阅", "refund",
8          "invoice"]
9
10     if req.intent == IntentCategory.TECHNICAL or any(kw in msg for kw in
11         technical_kws):
12         targets.append(AgentType.TECHNICAL)
13
14     if req.intent in (IntentCategory.BILLING, IntentCategory.ACCOUNT) or
15         any(kw in msg for kw in billing_kws):

```

```
11         targets.append(AgentType.BILLING)
12
13     deduped = list(dict.fromkeys(targets))
14     return [agent_type for agent_type in deduped if self._pool.get(agent_type)]
```

示例：

代码块

```
1  登录报错 401，而且这个月还重复扣款了
```

会触发：

代码块

```
1  TechnicalAgent + BillingAgent
```

性能路由

代码块

```
1  def _best_agent(self, agent_type: AgentType) -> Optional[BaseAgent]:
2      agents = self._pool.get(agent_type, [])
3      if not agents:
4          return None
5      return max(agents, key=lambda a: a.stats.routing_score())
```

8. routing_score 与 Monitor 降权

文件：[agents/agent_orchestrator.py](#)、[monitor/performance_monitor.py](#)

AgentStats

代码块

```
1  @dataclass
2  class AgentStats:
3      total: int = 0
4      success: int = 0
5      total_ms: float = 0.0
6      monitor_penalty: float = 0.0
```

```

7
8     def routing_score(self) -> float:
9         latency_score = 1.0 / (1.0 + self.avg_ms / 1000)
10        base_score = self.success_rate * 0.7 + latency_score * 0.3
11        return base_score * max(0.0, 1.0 - self.monitor_penalty)

```

Monitor 采集并写回 penalty

代码块

```

1  async def _collect(self) -> None:
2      agent_stats = self._orchestrator.get_stats()
3      tool_stats = self._tool_manager.get_stats()
4      routing_penalties: Dict[str, float] = {}
5
6      for agent_key, s in agent_stats.items():
7          sr = s["success_rate"]
8          ms = s["avg_ms"]
9          ...
10         routing_penalties[agent_key] = self._routing_penalty(sr, ms)
11
12     updater = getattr(self._orchestrator, "update_routing_penalties", None)
13     if updater:
14         updater(routing_penalties)

```

降权规则

代码块

```

1  @staticmethod
2  def _routing_penalty(success_rate: float, avg_ms: float) -> float:
3      penalty = 0.0
4      if success_rate < 0.90:
5          penalty += min(0.5, (0.90 - success_rate) * 2)
6      if avg_ms > 3000:
7          penalty += min(0.4, (avg_ms - 3000) / 10000)
8      return min(penalty, 0.9)

```

写回 Orchestrator

代码块

```

1  def update_routing_penalties(self, penalties: Dict[str, float]) -> None:
2      for agent_type, agents in self._pool.items():

```

```

3         for i, agent in enumerate(agents):
4             key = f"{agent_type.value}_{i}"
5             penalty = penalties.get(key, 0.0)
6             agent.stats.monitor_penalty = min(max(penalty, 0.0), 0.9)

```

这是真正的 Monitor 闭环：Monitor 不只是展示指标，而是会影响下一次 `_best_agent()` 的路由选择。

9. LLM-as-Judge 端到端评测

文件：evaluation/[evaluator.py](#)

真实调用 Orchestrator

代码块

```

1  orch_req = OrcReq(
2      message=question,
3      user_id=user_id,
4      conv_id=conv_id,
5      context=context,
6      history=history[-6:] if history else None,
7  )
8  orch_result = await self._orchestrator.run(orch_req)
9  actual_answer = orch_result.response

```

评测不是 mock 回复，而是真正走 Agent 编排链路。多轮用例会复用同一个 `conv_id`，并把前序轮次拼成评测历史上下文。

四维度评分

代码块

```

1  return QualityScores(
2      relevance=float(data.get("relevance", 0.5)),
3      accuracy=float(data.get("accuracy", 0.5)),
4      completeness=float(data.get("completeness", 0.5)),
5      helpfulness=float(data.get("helpfulness", 0.5)),
6  )

```

维度	含义

relevance	是否直接回应用户问题
accuracy	信息是否准确
completeness	是否完整解决需求
helpfulness	用户能否据此行动

回归检测

代码块

```
1 def _detect_regressions(self, current: Dict[str, float]) -> List[str]:
2     prev_report = self._history[-1] if self._history else self._baseline
3     if prev_report is None:
4         return []
5     prev = prev_report.avg_scores
6     regressions = []
7     for metric, value in current.items():
8         if metric in prev and prev[metric] > 0:
9             delta = (value - prev[metric]) / prev[metric]
10            if delta < -0.05:
11                regressions.append(
12                    f"{metric}: {prev[metric]:.3f} → {value:.3f} (退化
13                    {abs(delta):.1%})"
14                    )
15            return regressions
```

当前评测支持内存中的上一轮结果，也支持通过 EVAL_BASELINE_PATH 持久化基线。Docker Compose 默认把基线保存到 /app/data/eval/baseline.json，宿主机可在 ./data/eval/baseline.json 查看。

10. Unicode 安全处理

文件: [agents/agent_orchestrator.py](#)

代码块

```
1 async def _call_llm(self, req: Request) -> str:
2     def _clean(s: str) -> str:
3         return s.encode("utf-8", errors="ignore").decode("utf-8")
4
5     messages = []
6     if req.context:
```

```
7         messages.append({"role": "user", "content": f"[背景信
      息]\n{_clean(req.context)}"})
8         messages.append({"role": "assistant", "content": "好的，我已了解背景信
      息。"})
9         messages.append({"role": "user", "content": _clean(req.message)})
```

ChromaDB 或 Redis 中可能存储包含 Unicode 代理字符的数据。_clean() 会在调用 LLM 前移除无法 UTF-8 编码的字符，避免 API 请求崩溃。

11. 环境自适应：官方 API 与第三方兼容 API

文件: core/intent_recognizer.py、api/main.py

配置读取：

代码块

```
1 def _anthropic_cfg() -> Dict[str, Any]:
2     key = os.getenv("ANTHROPIC_API_KEY", "")
3     if not key:
4         raise RuntimeError("未设置 ANTHROPIC_API_KEY")
5     cfg = {
6         "api_key": key,
7         "model": os.getenv("ANTHROPIC_MODEL", "claude-3-5-sonnet-20241022"),
8     }
9     base_url = os.getenv("ANTHROPIC_BASE_URL", "").strip()
10    if base_url:
11        cfg["base_url"] = base_url
12    return cfg
```

意图识别中根据 base_url 自动切换策略：

代码块

```
1 self._embedding_enabled = not bool(base_url)
```

模式	条件	行为
官方 Anthropic	ANTHROPIC_BASE_URL 为空	三路融合：LLM + Embedding + Pattern
第三方兼容 API		两路融合：LLM + Pattern

	ANTHROPIC_BASE_URL 有值	
--	-----------------------	--

调用层不需要关心差异，只需要通过环境变量切换。